

Creational Design Patterns

Chapter 02 - Sub-Chapter 02.02

How objects are created -- safely, flexibly, and efficiently

Singleton	Only one instance ever exists
Factory Method	Let subclasses decide which object to create
Abstract Factory	Family of related objects without specifying concrete classes
Builder	Build a complex object step by step
Prototype	Clone an existing object instead of creating from scratch

1 Singleton

One instance, globally accessible

The Singleton pattern ensures a class has exactly **one instance** throughout the entire application and provides a global access point to it. It is used for shared, expensive resources like loggers, config managers, and DB connection pools. Java offers several implementation approaches -- each with different trade-offs around thread safety, lazy initialisation, and serialisation safety.

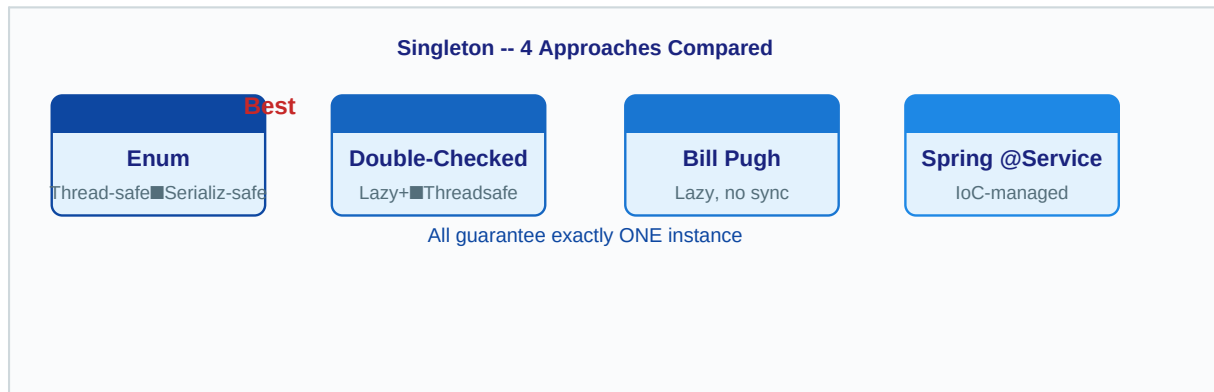


Figure: Four Singleton approaches -- all guarantee one instance. Enum is the recommended choice.

Example 1 — Enum Singleton (Recommended)

The JVM guarantees enum constants are instantiated exactly once. This is automatically thread-safe and immune to reflection and serialisation attacks. Use this whenever you need a Singleton in Java.

```
// 1. Enum Singleton (BEST -- thread-safe, serialization-safe)
enum Database {
    INSTANCE;
    private String url = "jdbc:mysql://localhost/mydb";
    public void query(String sql) {
        System.out.println("Running on " + url + ": " + sql);
    }
}
// Usage
Database.INSTANCE.query("SELECT * FROM users");
System.out.println(Database.INSTANCE == Database.INSTANCE); // true
```

Example 2 — Double-Checked Locking

Lazy initialisation: the instance is only created when first needed. Uses **volatile** to prevent CPU instruction reordering and **synchronized** only on the first creation (not every call), making it efficient in multi-threaded environments. Common in legacy codebases.

```
// 2. Double-Checked Locking (thread-safe lazy init)
class Logger {
    private static volatile Logger instance;
    private Logger() {}
    public static Logger getInstance() {
        if (instance == null) {
            synchronized (Logger.class) {
                if (instance == null)
                    instance = new Logger();
            }
        }
        return instance;
    }
}
```

```

    public void log(String msg) { System.out.println("[LOG] " + msg); }
}
// Usage
Logger.getInstance().log("App started");
Logger.getInstance().log("Request received");

```

Example 3 — Bill Pugh / Static Inner Class

Leverages Java's class loader guarantee: the inner **Holder** class is only loaded when **getInstance()** is first called. No synchronised block needed — the JVM handles thread safety during class loading. This is the cleanest lazy singleton without any performance overhead.

```

// 3. Bill Pugh (static inner class -- lazy, thread-safe, no sync)
class ConfigManager {
    private String env = "production";
    private ConfigManager() {}
    private static class Holder {
        static final ConfigManager INSTANCE = new ConfigManager();
    }
    public static ConfigManager getInstance() { return Holder.INSTANCE; }
    public String getEnv() { return env; }
}
// Usage
System.out.println(ConfigManager.getInstance().getEnv()); // production
// Inner class Holder is loaded ONLY when getInstance() is first called.

```

Example 4 — Spring @Service (IoC-managed Singleton)

In Spring applications you rarely implement Singleton yourself. Any **@Component**, **@Service**, or **@Repository** bean is a Singleton by default — Spring's IoC container creates it once and injects the same object everywhere it is autowired. This is the preferred approach in modern Spring apps.

```

// 4. Spring Bean Singleton (default scope -- managed by IoC container)
@Service // Spring creates ONE instance
public class NotificationService {
    public void send(String msg) {
        System.out.println("Sending: " + msg);
    }
}
// Two places autowire the same bean -- same object, different reference names
@Autowired NotificationService svc1;
@Autowired NotificationService svc2;
System.out.println(svc1 == svc2); // true

```

Singleton Approaches — Comparison

Approach	Thread-Safe	Lazy Init	Serialization Safe	Best For
Enum	Yes (JVM)	No (eager)	Yes	Most Java cases
Double-Checked Lock	Yes (volatile)	Yes	Manual effort	Legacy, no Spring
Bill Pugh	Yes (JVM CL)	Yes	Manual effort	Pure Java, clean
Spring @Service	Yes (IoC)	Yes (lazy opt)	N/A	Spring Boot apps

Key Rules

- Prefer Enum Singleton in plain Java -- thread-safe and serialization-safe with zero boilerplate.
- In Spring Boot apps, use `@Service/@Component` -- the container manages the singleton lifecycle.
- Double-Checked Locking requires 'volatile' -- omitting it causes subtle multi-threading bugs.
- Bill Pugh pattern is ideal for lazy init in non-Spring codebases without any synchronised block.
- Never use Singleton for per-user state (sessions, shopping carts) -- use prototype/request scope instead.
- Test carefully: Singleton state persists between tests and can cause hidden test coupling.

Q1. Why is Enum the safest Singleton in Java?

Java guarantees enum values are created once by the JVM -- this is automatically thread-safe and protected against reflection attacks and serialisation/deserialisation creating new instances.

Q2. What is the difference between Enum and Double-Checked Locking Singleton?

Enum is eager (created at class load), always thread-safe, and serialization-safe with no extra code. DCL is lazy (created on first use), thread-safe only with volatile, and needs manual serialization handling.

Q3. Why does Bill Pugh not need a synchronised block?

The JVM guarantees that class loading is thread-safe. The inner Holder class is only loaded when `getInstance()` is first called, so the INSTANCE field is initialised exactly once without locking.

Q4. How does Spring make a bean a Singleton?

Spring's IoC container creates the bean once when the application context starts and stores it. Every `@Autowired` injection point receives the same object reference.

Q5. When should you NOT use Singleton?

Avoid it for objects with per-user or per-request mutable state (shopping carts, HTTP sessions). Use Spring's request or session scope, or prototype scope, instead.

2 Factory Method

Delegate object creation to a factory

The Factory Method pattern centralises object creation in a single place -- a factory -- so the rest of the code never calls 'new' directly on concrete classes. When you ask `ShapeFactory.create("circle")` for a shape, you get back a `Circle` without knowing or caring about the constructor details. This makes it easy to add new shapes later (just add a new class and one line in the factory) without touching any calling code.

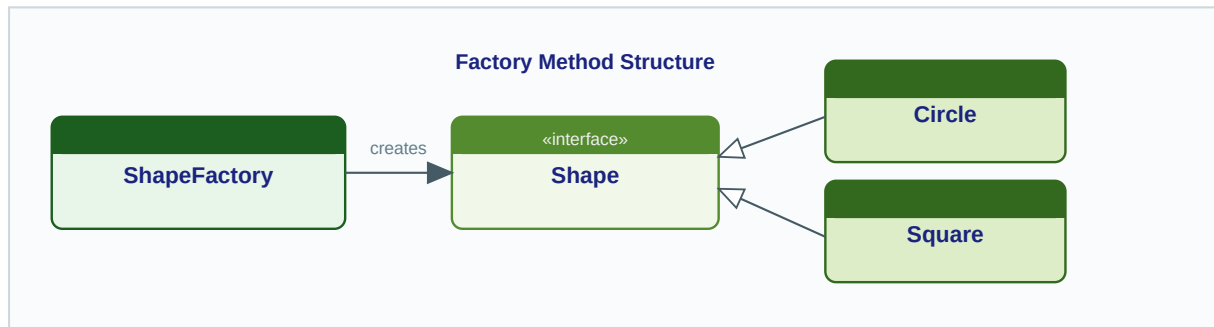


Figure: `ShapeFactory` creates objects that implement the `Shape` interface; `Circle` and `Square` are concrete implementations.

Java Example

```
interface Shape { void draw(); }
class Circle implements Shape {
    public void draw() { System.out.println("Drawing Circle"); }
}
class Square implements Shape {
    public void draw() { System.out.println("Drawing Square"); }
}
class ShapeFactory {
    static Shape create(String type) {
        if (type.equals("circle")) return new Circle();
        if (type.equals("square")) return new Square();
        throw new IllegalArgumentException("Unknown: " + type);
    }
}
// Usage
Shape s1 = ShapeFactory.create("circle");
Shape s2 = ShapeFactory.create("square");
s1.draw(); // Drawing Circle
s2.draw(); // Drawing Square
```

Key Rules

- Program to the interface (`Shape`), never to the concrete class (`Circle`).
- The factory method is the single place to change when adding a new type.
- Throw a clear exception for unknown types rather than returning null.
- Keep the factory method static when no instance state is needed.

Q1. What is the core benefit of Factory Method?

Calling code depends only on the `Shape` interface, not on `Circle` or `Square`, so adding new shapes requires no changes outside the factory.

Q2. How do you add a Triangle shape?

Create a `Triangle` class implementing `Shape`, then add one if-branch in `ShapeFactory.create()` -- no other code needs to change.

Q3. What happens if an unknown type is requested?

The factory throws `IllegalArgumentException`, giving a clear error message instead of silently returning null.

Q4. How does Factory Method differ from directly calling new?

Direct new couples the caller to a concrete class. Factory Method decouples creation from use, enabling easier testing and extension.

Q5. Can the factory return different subtypes at runtime?

Yes -- the factory can inspect any runtime condition (config, feature flag, user role) and return the appropriate subtype transparently.

3 Abstract Factory

Create families of related objects

Abstract Factory extends the Factory Method idea to a whole family of related objects. Instead of one factory method for one product, you define a factory interface whose implementations produce an entire consistent set of products. In the UI example, UIFactory has createButton(). LightTheme always produces light-styled widgets; DarkTheme always produces dark-styled ones. Swapping the factory swaps the entire theme consistently -- you can never accidentally mix a DarkButton into a LightTheme.

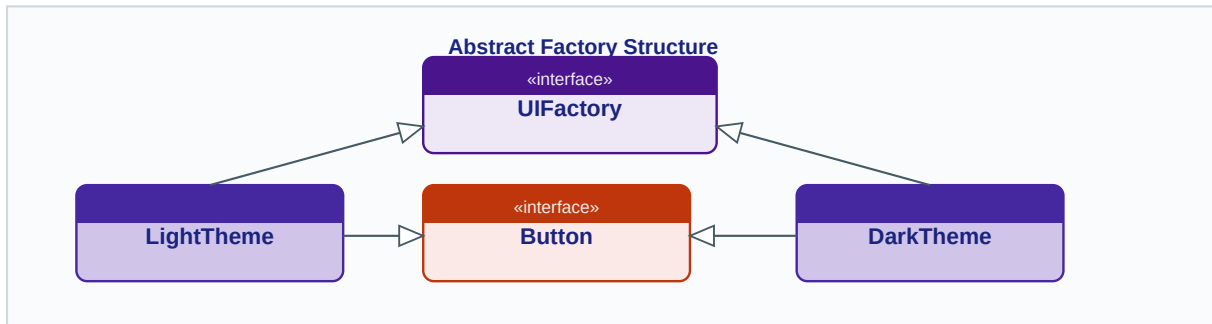


Figure: UIFactory interface implemented by LightTheme and DarkTheme; Button interface implemented by LightButton and DarkButton.

Java Example

```
interface Button { void render(); }
class LightButton implements Button {
    public void render() { System.out.println("Light Button"); }
}
class DarkButton implements Button {
    public void render() { System.out.println("Dark Button"); }
}
interface UIFactory { Button createButton(); }
class LightTheme implements UIFactory {
    public Button createButton() { return new LightButton(); }
}
class DarkTheme implements UIFactory {
    public Button createButton() { return new DarkButton(); }
}
// Usage
UIFactory factory = new DarkTheme();
Button btn = factory.createButton();
btn.render(); // Dark Button
// Swap entire theme: factory = new LightTheme();
```

Key Rules

- The factory interface guarantees consistency -- all products come from the same family.
- Swap the entire product family by replacing the factory object, not individual calls.
- Add a new theme by creating a new UIFactory implementation; existing code unchanged.
- Abstract Factory is ideal when products must be used together and must match.

Q1. How does Abstract Factory differ from Factory Method?

Factory Method creates one product; Abstract Factory creates a family of related products that are designed to work together.

Q2. How do you switch from dark to light theme?

Simply assign `factory = new LightTheme()`. Every subsequent `createButton()` call automatically returns a `LightButton`.

Q3. What guarantees consistency across products?

All products are created by the same concrete factory class, so they always belong to the same family by construction.

Q4. When would you add a `createTextField()` method?

When `TextField` is another product in the UI family -- add the method to `UIFactory` and implement it in both `LightTheme` and `DarkTheme`.

Q5. What is the main drawback of Abstract Factory?

Adding a new product type (e.g., `createCheckbox()`) requires changing every existing factory implementation, which can be disruptive.

4 Builder

Construct complex objects step by step

The Builder pattern solves the 'telescoping constructor' problem -- when an object has many optional fields, constructors with many parameters become unreadable and error-prone. Builder lets you set only the fields you care about using a fluent chain of method calls, then call `build()` to receive the final object. The Pizza example shows three optional fields set in any order; the resulting object is only created at the end.

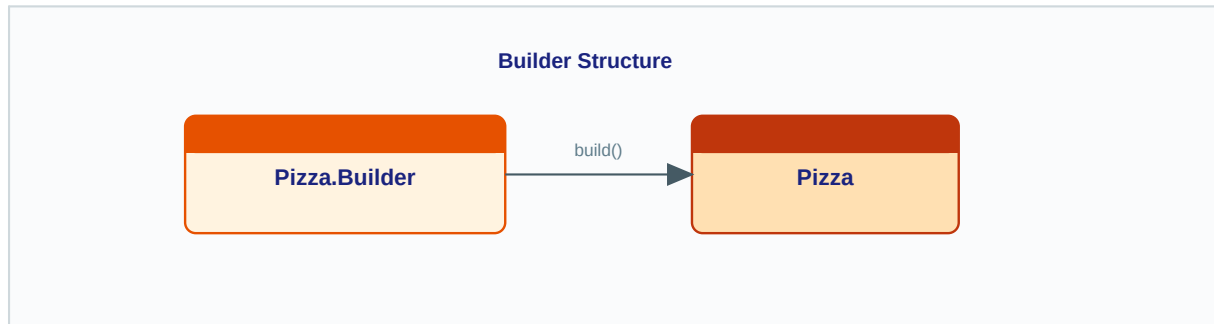


Figure: `Pizza.Builder` accumulates configuration through chained calls and produces a `Pizza` object when `build()` is called.

Java Example

```
class Pizza {
    String size; String crust; String topping;
    private Pizza() {}
    static class Builder {
        private Pizza pizza = new Pizza();
        Builder size(String s)    { pizza.size=s;    return this; }
        Builder crust(String c)  { pizza.crust=c;    return this; }
        Builder topping(String t) { pizza.topping=t; return this; }
        Pizza build() { return pizza; }
    }
    public String toString() {
        return size + " pizza, " + crust + " crust, " + topping;
    }
}
// Usage
Pizza p = new Pizza.Builder()
    .size("Large")
    .crust("Thin")
    .topping("Cheese")
    .build();
System.out.println(p); // Large pizza, Thin crust, Cheese
```

Key Rules

- Return 'this' from each setter to enable method chaining (fluent API).
- Call `build()` only once at the end -- it produces the final, complete object.
- Builder is ideal when an object has more than 3-4 optional parameters.
- Make the outer class constructor private to force use of the Builder.

Q1. What problem does Builder solve?

It replaces constructors with many parameters (which are hard to read) with a readable chain of named setter calls.

Q2. Why does each Builder method return 'this'?

Returning 'this' allows chaining: `.size("L").crust("Thin").build()` on one line, which is more readable than multiple separate statements.

Q3. Can you omit some fields when using Builder?

Yes -- fields not set simply keep their default values (null or 0), unlike constructors where all parameters must be supplied.

Q4. How does Builder differ from Factory Method?

Factory Method picks which type to create; Builder constructs one type step by step, letting you vary the internal configuration.

Q5. When is Builder overkill?

For simple objects with one or two required fields, a plain constructor or static factory method is simpler and easier to read.

5 Prototype

Copy an existing object instead of building from scratch

The Prototype pattern creates new objects by cloning an existing one rather than constructing from scratch. This is useful when object initialisation is expensive (reading config files, connecting to services) and you need many similar objects with small variations. In the example, AppConfig is loaded once; each subsequent environment just clones the original and tweaks the port -- no re-reading the config file needed.

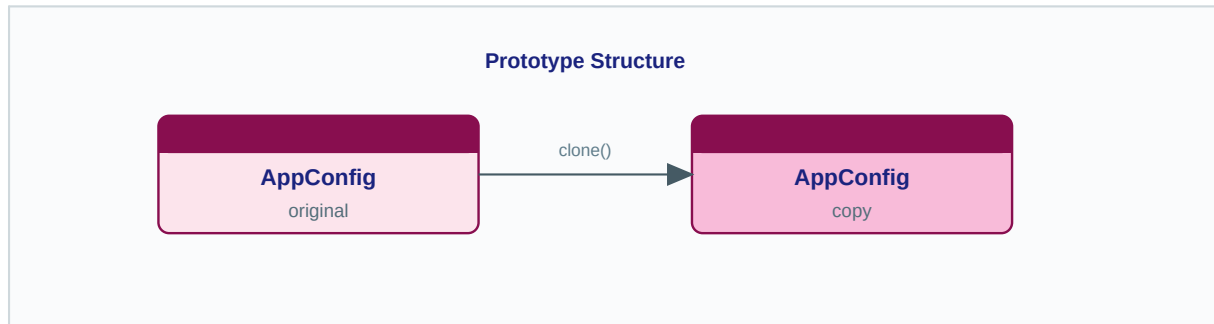


Figure: AppConfig.clone() produces an independent copy; changes to the copy do not affect the original.

Java Example

```
class AppConfig implements Cloneable {
    String host; int port;
    AppConfig(String host, int port) { this.host=host; this.port=port; }
    public AppConfig clone() {
        try { return (AppConfig) super.clone(); }
        catch (CloneNotSupportedException e) { throw new RuntimeException(e); }
    }
    public String toString() { return host + ":" + port; }
}
// Usage
AppConfig original = new AppConfig("localhost", 8080);
AppConfig copy     = original.clone();
copy.port = 9090;
System.out.println(original); // localhost:8080
System.out.println(copy);     // localhost:9090
```

Key Rules

- Implement Cloneable and call super.clone() for a shallow copy.
- For deep copy (nested objects), manually clone each mutable field after super.clone().
- Prototype is most valuable when construction is expensive and variants differ minimally.
- Always verify that original and clone are independent -- modifying one must not affect the other.

Q1. What is the difference between shallow and deep copy?

Shallow copy copies field values (including references); deep copy also copies the objects those references point to, so no shared mutable state exists.

Q2. Why use Prototype instead of calling new?

Cloning skips expensive initialisation (DB queries, file reads) that ran once when the original was created.

Q3. Does changing copy.port affect original.port?

No -- int is a primitive, so it is copied by value. Both objects have independent port fields after clone().

Q4. When would you need a deep clone?

When AppConfig contains a List or Map field -- those are objects and both original and copy would share the same List unless you manually clone it.

Q5. How does Prototype relate to copy constructors?

A copy constructor achieves the same goal but requires the caller to know the concrete class. Prototype works through the Cloneable interface polymorphically.

Creational Patterns -- Quick Reference

Pattern	Intent	Key Class	Java Example	Use When
Singleton	One instance ever	enum Database	Database.INSTANCE E.query()	Shared resource (config, pool)
Factory Method	Centralise object creation	ShapeFactory	ShapeFactory.create ("circle")	Type varies at runtime
Abstract Factory	Family of related objects	UIFactory / LightTheme	factory.createButton ()	Switch whole product family
Builder	Step-by-step construction	Pizza.Builder	new Pizza.Builder(). size().build()	Many optional parameters
Prototype	Clone existing object	AppConfig (Cloneable)	original.clone()	Expensive init, small variants